

Evolution Simulator (Neuroevolution) — Full Project Plan

Goal: Build a Python artificial-life simulator in which creatures evolve physical traits first, then evolve their own neural-network "brains", and eventually coexist with evolving predators — all using a genetic algorithm you write yourself. No LLMs, no pre-trained models, no backpropagation.

This document gives you **direction only** — architecture, sequencing, concepts to learn, and where to learn them. No implementation code.

1. How to use this plan

Work through the phases **in order**. Each phase ends with something that runs and is visibly better/different than the last. Don't skip ahead to brains or predators before Phase 1-3 are solid — the whole point of the architecture below is that later phases *plug into* earlier ones without rewriting them.

Treat every phase like a small milestone you could screenshot or record a GIF of. That habit alone will make your eventual README/CV write-up much easier.

2. Packages you'll need (and when)

Package	Used for	Introduce in
<code>pygame</code>	Rendering the world, game loop, input	Phase 1
<code>numpy</code>	Vector maths, matrix operations for the neural net later	Phase 3 onward (light use), essential from Phase 6
<code>random</code> / <code>numpy.random</code>	Mutation, reproduction, spawning	Phase 3
<code>dataclasses</code> (stdlib)	Clean gene/config definitions	Phase 3
<code>matplotlib</code>	Graphs of fitness/speed/vision over generations	Phase 8
<code>pandas</code> (optional)	Easier logging/analysis of generation stats	Phase 8-9
<code>json</code> or <code>yaml</code> (stdlib / <code>pyyaml</code>)	Config files, saving/loading genomes	Phase 9

Package	Used for	Introduce in
<code>pytest</code>	Unit tests for genome/mutation/fitness logic	Phase 4 onward
<code>git</code> + GitHub	Version control, so new phases can't silently break old ones	Day 1

You do **not** need PyTorch, TensorFlow, or scikit-learn for this project. The neural network is small enough (tens to a couple hundred weights) that NumPy alone comfortably handles forward propagation. Avoiding a deep learning framework is actually a selling point — it proves you understand what's happening under the hood rather than calling `.fit()`.

3. Where to learn the underlying concepts

Don't binge tutorials that build "an evolution simulator" end-to-end — you said you don't want that, and it's the right call. Instead, learn the underlying *building blocks* independently, then assemble them yourself.

Pygame basics (game loop, drawing, input, FPS control)

- Official Pygame docs + the "Pygame Tutorial for Beginners" style content on freeCodeCamp's YouTube channel.
- You only need: window setup, the event loop, `draw.circle`/`draw.rect`, `clock.tick()`. Ignore sprites/animation systems for now.

Object-oriented Python (classes, inheritance, abstract base classes)

- Python docs on `abc` (Abstract Base Classes) — this is what will let you build the swappable `Brain` interface later.
- Corey Schafer's "Python OOP" YouTube series is a good structured refresher if Java has you thinking in a slightly different style than idiomatic Python.

Vectors and basic linear algebra with NumPy

- NumPy's own "Absolute Beginners" guide covers arrays, broadcasting, dot products — all you need for both movement vectors and later the neural network's matrix multiplications.

Genetic algorithms (selection, crossover, mutation, elitism)

- Search for "genetic algorithm" explainers that cover: fitness functions, selection strategies (roulette wheel, tournament, elitism), mutation rate, and crossover. Sebastian Lague's YouTube videos on evolution simulations are a good *conceptual* watch (don't copy his code/design — use it to understand *why* certain design choices exist).

- Focus specifically on: how do you avoid losing all genetic diversity (mutation too low) vs. never converging (mutation too high)?

Neural networks — the maths, not a framework

- 3Blue1Brown's "Neural Networks" YouTube series is the best conceptual grounding for forward propagation, weights, biases, and activation functions, without diving into backprop (which you don't need here).
- You specifically need to understand: what is a weight matrix, what is an activation function (e.g. tanh, sigmoid), and how do inputs flow through layers to produce outputs. You will **not** need to understand gradient descent/backpropagation for neuroevolution, since evolution replaces training.

Neuroevolution specifically

- Search "neuroevolution" and "evolving neural networks without backpropagation" — this is a real, published area of AI research (e.g. NEAT — NeuroEvolution of Augmenting Topologies, by Kenneth Stanley). You don't need to implement NEAT itself (it's more advanced — dynamically evolving network *structure*, not just weights), but reading about it will help you understand the space your project sits in, and gives you a genuinely good "further work" talking point for interviews.

Data visualisation

- Matplotlib's own "Quick Start" guide covers everything you need: line plots of a stat over generations, maybe a scatter plot of population traits.

4. Project layout (set this up before writing any simulation logic)

```

EvolutionSimulator/
|
├─ main.py                # entry point: sets up and runs the simulation
├─ config.py              # all tunable constants in one place
├─ requirements.txt
├─ README.md
|
├─ simulation/
|   └─ world.py            # the environment: boundaries, food list,
creature list
|   └─ creature.py         # Creature class (position, energy, age,
genome, brain)
|   └─ food.py             # Food class
|   └─ predator.py         # added in Phase 7
|   └─ genome.py           # Genome representation + mutation/crossover

```

```

logic
|   └─ brain/
|       └─ base_brain.py          # abstract Brain interface
|       └─ rule_based_brain.py   # Phase 1-5 hardcoded logic
|       └─ neural_brain.py       # Phase 6+ neural network brain
|       └─ evolution.py          # generation loop: fitness, selection,
reproduction
|
└─ analysis/
    └─ logger.py                 # records per-generation stats to disk
    └─ plots.py                  # generates graphs from logged data
|
└─ tests/                       # pytest unit tests for
genome/mutation/fitness
|
└─ data/
    └─ logs/
    └─ graphs/

```

Two design rules to set up from day one, because they're what let you add brains and predators later *without rewriting existing code*:

1. **The `Brain` class is an interface, not a concrete implementation.** Even in Phase 1 when your "brain" is just an `if food_visible: turn towards it` rule, wrap that logic inside a class that exposes a single `decide(inputs) -> outputs` method. The `Creature` should only ever call `self.brain.decide(...)` — it should never contain movement-decision logic itself. When you build the neural-network brain in Phase 6, it implements the exact same interface, so nothing else in the codebase needs to change.
2. **The `Genome` is the single source of truth.** A creature's physical traits *and*, later, its brain weights all live inside one `Genome` object. The creature's constructor reads from the genome to configure itself (speed, vision radius, energy efficiency, and — later — the weight matrices for its brain). Evolution (selection, mutation, crossover) only ever operates on genomes, never on live creature objects directly. This is what makes "adding a new gene" a small, localized change instead of a rewrite.

Keep `config.py` as the *only* place magic numbers live (population size, mutation rate, food spawn rate, world dimensions, energy costs). This turns "run an experiment with 3× the mutation rate" into changing one line, which matters a lot for Phase 9.

5. Phased roadmap

Phase 0 — Setup (half a day)

- Initialise a git repo and push to GitHub immediately, with commits from the start (this

itself is a small positive signal in interviews).

- Set up a virtual environment, `requirements.txt`, and the folder structure above (even with empty files).
- Get a blank Pygame window opening and closing cleanly with a working game loop and frame-rate cap. Nothing else.

Phase 1 — World & rendering

- Build the `World` class: dimensions, boundaries, a list of food, a list of creatures.
- Draw the world and a single static creature as a circle.
- Learn: Pygame drawing primitives, coordinate systems, the game loop.
- **Milestone:** a window shows a world with food and one creature on screen.

Phase 2 — Movement & energy (no evolution yet)

- Add a single hardcoded creature that moves (randomly at first), loses energy over time, and dies when energy hits zero.
- Add food: when a creature overlaps food, gain energy, food disappears and a new one spawns elsewhere.
- Learn: collision detection (distance/overlap checks), basic timers.
- **Milestone:** one creature wanders, eats, and eventually dies. This proves your core simulation loop works before any AI enters the picture.

Phase 3 — Physical genes & the evolutionary algorithm

- Introduce `Genome` with a handful of physical traits: speed, vision radius, turning speed, energy efficiency, size.
- Spawn a population (e.g. 50-100) instead of one creature, each with randomised starting genes.
- Implement a fitness function (e.g. food eaten + time survived – energy wasted).
- Implement selection (start simple: top N% survive and reproduce), mutation (small random change per gene, per generation), and the generation loop itself (all creatures die or a timer runs out → select → reproduce → mutate → repeat).
- Learn: selection strategies (elitism vs. tournament vs. roulette wheel — pick one, elitism is simplest to start), how to tune mutation magnitude so change is visible but not chaotic.
- **Milestone:** run 50+ generations and watch average speed/vision change over time. This is your first "real" evolutionary algorithm and is already a complete, demoable project.

Phase 4 — Refactor into a `Brain` interface (important, don't skip)

- Before touching neural networks, extract your current movement logic (e.g. "turn

toward visible food, else wander") into a `RuleBasedBrain` class implementing a shared `Brain` interface (`decide(inputs) -> outputs`).

- The `Creature` should now only ever call `self.brain.decide(...)` — it should not contain any if/else movement logic of its own.
- Learn: abstract base classes in Python, the general software-design idea of "programming to an interface, not an implementation."
- **Milestone:** identical behaviour to Phase 3, but the codebase is now structured so swapping the brain implementation is a one-line change. This phase produces no visible change in the simulation — that's the point. It's a structural investment for Phase 5–6.

Phase 5 — Wire in a (not-yet-evolved) neural network

- Build a small feedforward network by hand using NumPy: define inputs (e.g. distance/angle to nearest food, distance/angle to nearest wall, current energy), a hidden layer, and outputs (turn amount, move speed).
- First just hardcode or randomly initialise its weights and confirm a creature controlled by it moves sensibly (or chaotically — that's expected before evolution is involved).
- Learn: forward propagation by hand (weighted sum + activation function), matrix multiplication with NumPy, choosing an activation function (tanh is a common, well-behaved choice for movement outputs).
- **Milestone:** a `NeuralBrain` class exists, implements the same interface as `RuleBasedBrain`, and controls a creature — but its weights aren't evolving yet.

Phase 6 — Neuroevolution

- Extend `Genome` to also store the network's weights (a flat list of numbers is fine; the `NeuralBrain` reshapes them into weight matrices when constructed).
- Your existing mutation/selection/reproduction code from Phase 3 should now be mutating *both* physical genes and brain weights — this is where the earlier "genome as single source of truth" design pays off.
- Turn off the rule-based fallback; brains start random and must evolve useful behaviour from scratch.
- Learn: how genome length affects mutation impact (mutating 1 of 5 physical genes is a big change; mutating 1 of 100 weights is small — you may want a different mutation rate for weights vs. physical traits).
- **Milestone:** creatures with literally random initial brains gradually evolve competent foraging behaviour over generations, with no hardcoded logic telling them how. This is the core "wow" moment of the project.

Phase 7 — Predators (optional but strong addition)

- Add a `Predator` class, structurally similar to `Creature` (its own genome, its own brain, its own fitness function based on successful hunts rather than foraging).

- Give prey an additional input (distance/angle to nearest predator) so their brain has the information needed to evolve avoidance behaviour.
- Learn: how to design two competing fitness functions so that neither species trivially "wins" (e.g. predators too fast wipes out prey entirely and the simulation collapses — you'll want to tune population sizes, reproduction rates, and energy costs).
- **Milestone:** an evolutionary arms race — prey evolving evasion, then predators adapting, back and forth over generations.

Phase 8 — Statistics & visualisation

- Log per-generation stats (avg fitness, avg speed, avg vision, average survival time, population size) to a CSV or JSON file each run.
- Build a separate script (`analysis/plots.py`) that reads the log and produces matplotlib graphs — generation vs. average trait, generation vs. population, etc.
- Learn: matplotlib line plots, basic pandas grouping if you use it for the logs.
- **Milestone:** you can point to a graph and say "here's where predators were introduced and prey vision jumped."

Phase 9 — Configuration & experiments

- Move every tunable value (mutation rate, population size, food density, world size, energy costs, network hidden-layer size) into (`config.py`) or a YAML/JSON config file.
- Run structured experiments: vary one variable at a time (mutation rate, food scarcity, predator count) and compare the resulting graphs.
- Write up your observations — this turns the project from "I built a simulation" into "I ran experiments and found X."
- **Milestone:** 3–4 short written experiment summaries with accompanying graphs, ready to drop into your README or a short report.

Phase 10 — Polish

- README with: project description, GIF/screenshot, how to run it, architecture overview, and a "results/experiments" section.
- Clean up (`config.py`) documentation, add a few (`pytest`) tests around genome mutation and fitness calculation (these are pure functions and easy to test — good for showing testing discipline).
- Optional: a simple live control panel (sliders for mutation rate/food density while running) if you have time left — this is a stretch goal, not a requirement.

6. Stretch goals (only after Phase 6+ is solid)

Pick at most one or two — these are for if you finish early or want the project to keep

growing after your placement deadline:

- **Sexual reproduction:** children inherit genes from two parents (crossover) instead of cloning + mutating one parent.
 - **Multiple species:** two prey types with different starting traits, observe whether they specialise into different niches.
 - **Seasons/weather:** food density changes over time, forcing brains to adapt to scarcity.
 - **NEAT-style topology evolution:** instead of a fixed network shape, let network structure itself evolve (this is genuinely advanced and a strong "further work" item to mention even if you don't fully build it).
 - **Live brain visualisation:** draw the neural network's neurons/weights for a selected creature, updating as it moves.
-

7. Keeping earlier phases from breaking

Since the whole plan depends on each phase building cleanly on the last:

- Commit to git after every phase, not just at the end. Tag or branch before starting a risky refactor (like Phase 4) so you can always roll back.
- Write a handful of `pytest` tests around genome mutation and fitness calculation as soon as they exist (Phase 3) — these are pure functions with no Pygame/rendering dependency, so they're cheap to test and will immediately tell you if a later change (like adding brain weights to the genome in Phase 6) silently breaks physical-trait mutation.
- Whenever you're tempted to add a special case directly inside `Creature` or `World`, stop and ask whether it belongs in `Genome` or `Brain` instead — that discipline is what keeps Phase 7 (predators) from requiring a rewrite of Phase 3–6.